TP 3 – Modélisation UML

Système de gestion d'une résidence universitaire

Diagramme de cas d'utilisation · Diagramme de classes · Diagramme de séquence · Diagramme d'états-transitions

1) Diagramme de cas d'utilisation

Le système est utilisé par deux acteurs : le **Responsable de la résidence**, qui pilote toutes les fonctionnalités, et l'**Étudiant**, à l'origine de la demande d'hébergement.

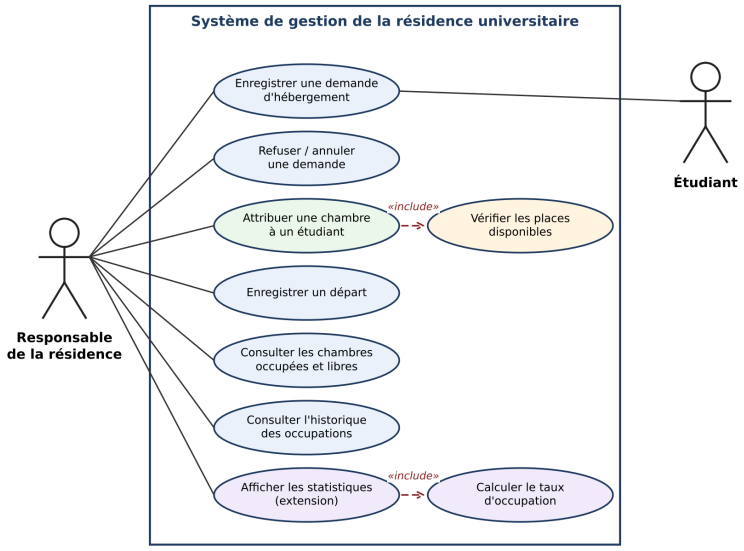


Figure 1 – Cas d'utilisation du système

Cas d'utilisation	Acteur(s)	Rôle
Enregistrer une demande d'hébergement	Étudiant, Responsable	Crée une demande à l'état EN_ATTENTE
Refuser / annuler une demande	Responsable	Clôture une demande en attente (REFUSEE ou ANNULEE)
Attribuer une chambre à un étudiant	Responsable	Crée l'attribution et occupe une place ; *inclut* la vérification des places
Vérifier les places disponibles	(inclus)	Garantit qu'aucune chambre pleine n'est attribuée
Enregistrer un départ	Responsable	Clôture l'attribution et libère la place
Consulter les chambres occupées et libres	Responsable	Visualise l'état du parc de chambres
Consulter l'historique des occupations	Responsable	Liste toutes les attributions, passées et en cours
Afficher les statistiques (extension)	Responsable	Affiche le taux d'occupation (*inclut* son calcul)

Relations : « Attribuer une chambre » *inclut* « Vérifier les places disponibles » (vérification obligatoire à chaque attribution) ; « Afficher les statistiques » *inclut* « Calculer le taux d'occupation ».

2) Diagramme de classes

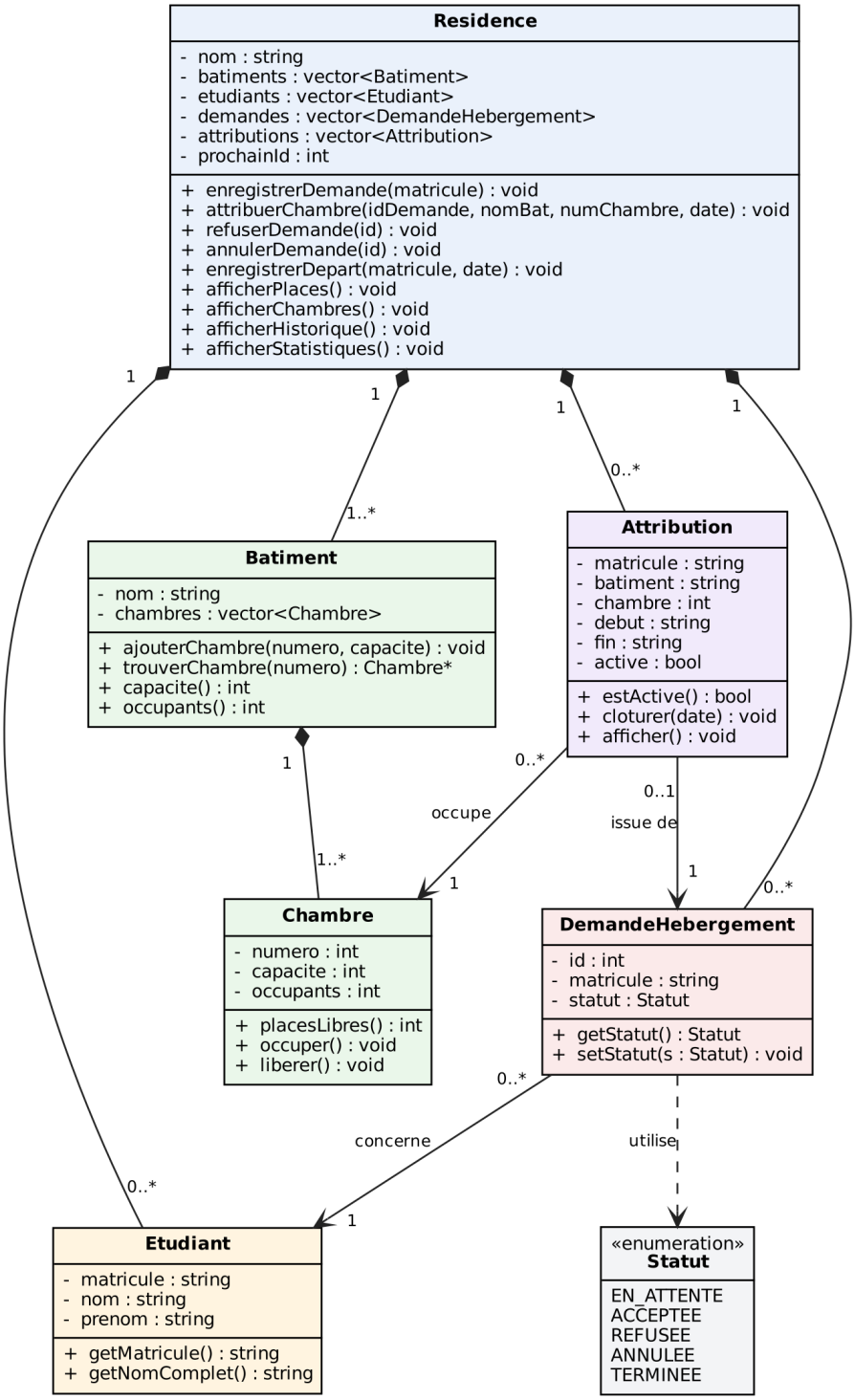


Figure 2 – Diagramme de classes (visibilités : - privé, + public)

Relations : la **Residence** est composée de bâtiments, d'étudiants, de demandes et d'attributions (composition, losange plein) ; un **Batiment** est composé de chambres. Une **DemandeHebergement** concerne un **Etudiant** ; une **Attribution** est issue d'une demande et occupe une **Chambre**. L'énumération **Statut** décrit le cycle de vie de la demande.

3) Diagramme de séquence : attribution d'une chambre

Le Responsable demande l'attribution ; la **Residence** orchestre les vérifications avant toute modification, afin d'éviter les affectations multiples.

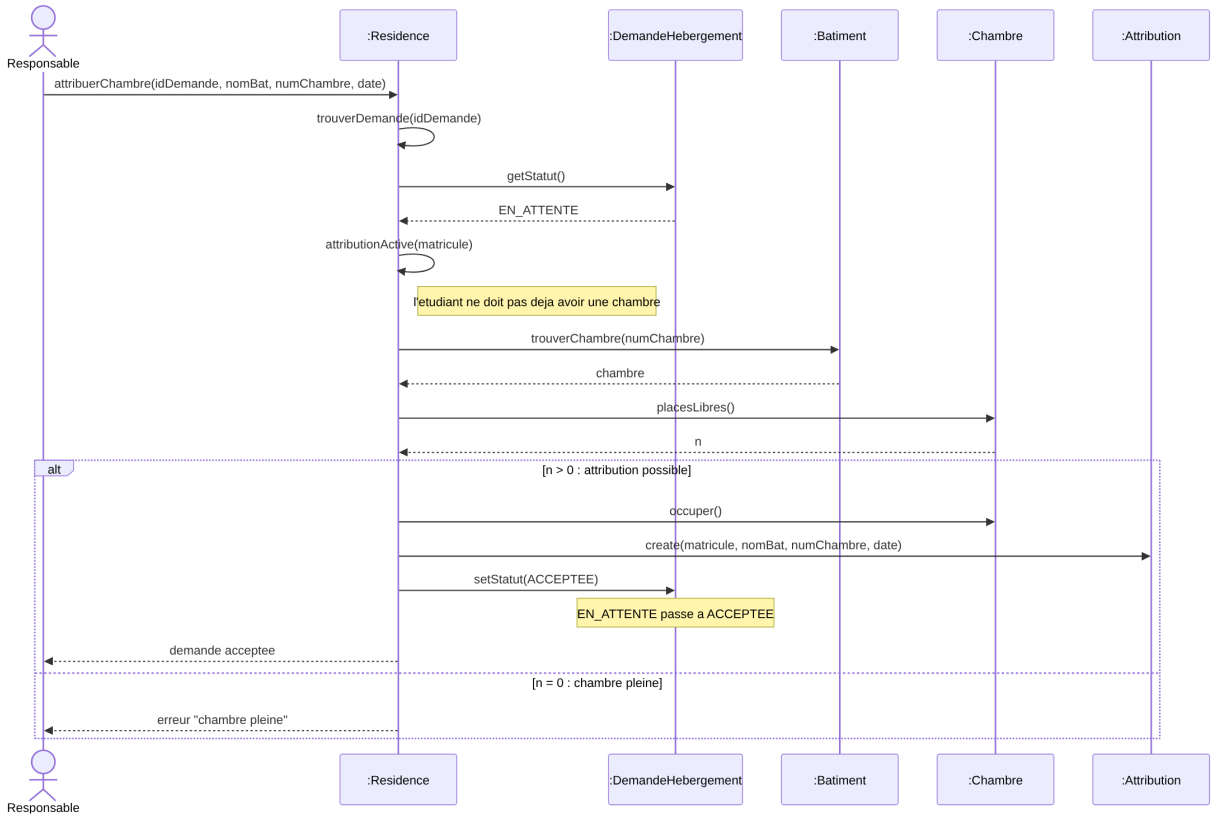


Figure 3 – Séquence « attribuerChambre »

Déroulement : la demande est retrouvée et doit être *en attente* ; l'étudiant ne doit pas déjà avoir une chambre ; la chambre est retrouvée dans le bâtiment ; on vérifie qu'il reste une place. **Si oui :** la chambre est occupée, l'attribution est créée et la demande passe à ACCEPTEE. **Sinon :** une erreur « chambre pleine » est affichée et rien n'est modifié.

4) Diagramme d'états-transitions : demande d'hébergement

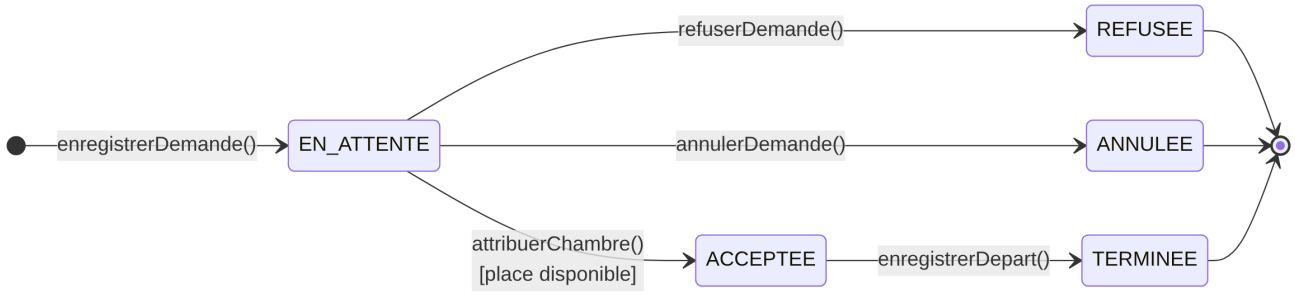


Figure 4 – Cycle de vie d'une demande d'hébergement

État	Signification	Transitions possibles
EN_ATTENTE	Demande déposée, pas encore traitée	→ ACCEPTEE, REFUSEE ou ANNULEE
ACCEPTEE	Chambre attribuée, l'étudiant est logé	→ TERMINEE (au départ)
REFUSEE	Demande rejetée par le responsable	état final
ANNULEE	Demande retirée	état final
TERMINEE	Occupation terminée, départ enregistré	état final

Seule une demande **EN_ATTENTE** peut être acceptée, refusée ou annulée, et seule une demande **ACCEPTEE** passe à TERMINEE lors du départ. Ces règles sont vérifiées dans les méthodes `attribuerChambre()`, `refuserDemande()`, `annulerDemande()` et `enregistrerDepart()` de la classe `Residence`.